COMP 208
Fall Semester 2025

INSTRUCTORS: Professor Michael Langer and DR. Chad Zammar

Assignment 3: Exploring Lists, IO, and dictionaries

Posted: October 30, 2025
Due date: November 13, 2025, 11:59 PM.

Before you start:

- Use your IDE (Thonny) to write and debug your solution. Only use Ed Lessons to submit the code.
- Use only Python material covered in the videos or in in class exercises up to and including lecture 16 (dictionaries), unless otherwise specified.
- If you get stuck, please see a T.A. or an instructor during office hours. When you come to office hours, please do not ask something like “Here is my program. What’s wrong with it?” Instead, try first to narrow down where the error is occurring and be ready to explain roughly what the problem is. We strongly encourage you to try to use the debugger.
- Policy on Academic Integrity: The assignments are to be done individually. While we encourage you to discuss the assignment with each other verbally, you should not tell each other how to solve the problem, and under no circumstances should you share your code or use Generative AI tools to generate it. See the Course Outline Sec. 6 for the policies. (You are expected to read that document.) If you have any questions, then please ask.
- Late policy: k*10 points out of 100 will be deducted if you submit k days of delay. So one day late means 10 points, two days late means 20 points, etc, up to 4 days late which is 40 points. Assignments will not be accepted after 4 days late.

- Note that submitting one minute late is equivalent to submitting 23 hours and 59 minutes late, etc. So, make sure you are nowhere near that threshold when you submit.

Assignment 3 learning objectives

- Learn to process and analyze real-world information using list slicing, indexing, and mathematical operations on nested structures.
- Learn to read information from CSV files and handle file input/output operations including parsing structured formats.
- Learn to write information to a CSV file.
- Learn to create copies of structures with nested elements, understanding how to preserve original information while working with modified versions for analysis.
- Develop skills in handling edge cases and designing robust processing workflows that can handle real-world challenges.
- Learn to implement proper error handling and raise appropriate exceptions (runtime errors) when encountering invalid input or unexpected conditions in data processing workflows.

No docstring is required for this assignment.

Smartwatch Step Data Analyzer

A fitness company collects per-minute step counts from users' smartwatches. Each day is recorded as a list of 1440 integers (step counts each minute). The product team wants a small analysis tool that extracts useful summaries for a week (7 days). The per-minute data for the week is available in a file called step_data.csv.

Write a Python function **smartwatch_analyzer** that takes a CSV filename as argument and computes the following using helper functions that rely on list slicing and indexing:

Core Analysis Function:

The function **smartwatch_analyzer** should:

- Take the CSV file name as argument
- Read the CSV file using **read_raw_data**
- Call all helper functions with the week data
- Compile results into the final report dictionary; details of this dictionary will be given after the helper functions are described
- Write data analysis to a CSV file
- Return the dictionary

Helper Functions:

1. Data Input:

You need to create a helper function called **read_raw_data** that reads step data from a CSV file. The function should take a filename as an argument and return a list of 7 lists, where each inner list contains 1440 integers representing minute-by-minute step counts for one day. Each line in the CSV contains 1440 comma-separated integers (one per minute of the day), and you should assume the CSV file is in the same directory as your Python code, and it is a valid file containing valid input. The function should be called like this: `read_raw_data('step_data.csv')` as in the example below, and return the `week_data` structure needed by the other functions.

Example usage:

```
week_data = read_raw_data('step_data.csv')
# Day 0
# week_data[0] = [step_count_minute_0, step_count_minute_1, ...,
step_count_minute_1439]
# Day 1
# week_data[1] = [step_count_minute_0, step_count_minute_1, ...,
step_count_minute_1439]
# ... and so on for all 7 days
```

2. Daily active window:

For each day, find the contiguous 60-minute window with the highest total steps (i.e., the most active hour). Return the start minute index (0–1439) and the total steps in that window for every day.

This helper function should be called **get_best_hour.** It takes `week_data` as an argument, and it returns a list of 7 tuples, where each tuple contains (start_minute_index, total_steps) for the best 60-minute window of that day.

Important consideration: The 60-minute window can span across midnight. For example, if the best hour is from 11:30 PM to 12:30 AM, this would involve:

- Minutes 1410-1439 from the current day (11:30 PM - 11:59 PM)
- Minutes 0-29 from the next day (12:00 AM - 12:30 AM), except for the last day of the week.

smartwatch_analyzer should call **get_best_hour** once and receive all 7 results. The final report (described later) should include the best hour information for every single day (7 results total).

3. Morning vs evening comparison:

For each day, compute total steps in the "morning" (06:00–11:59, minutes 360–719) and in the "evening" (17:00–21:59, minutes 1020–1319). Note that we are ignoring afternoon, for simplicity, and that our evening category ends well before midnight.

The function should be called **compare_morning_evening.** It takes `week_data` as an argument, and it returns a list of 7 tuples, where each tuple contains (`morning_total, evening_total, winner`) where `winner` is 'morning', 'evening', or 'equal'.

smartwatch_analyzer should call **compare_morning_evening**. The final report should include the morning/evening comparison for all 7 days.

4. Peak-day subsequence:

Identify the day with the highest single-minute step count across the week. From that day, return a 15-minute subsequence centered on that peak minute (7 minutes before, the peak minute, and 7 minutes after). If the peak is within 7 minutes of day start or end, pad the missing positions with zeros so the returned subsequence is always length 15.
The function should be called **extract_centered_subsequence.** It takes `week_data` as an argument, and it returns a tuple containing (`day_index`, `peak_minute, subsequence`) where:

- day_index is the day (0-6) where the peak occurred

- peak_minute is the minute index (0-1439) where the peak occurred.
- subsequence is a list of exactly 15 integers centered on the peak minute

For example if the peak is located on the 3rd day 5 minutes into the day the output might look something like: (2, 4, [0,0,0,30,28,25,31,39,26,22,18,19,12,10,8])

5. Hourly Averages:

For each day produce a list of 24 hourly averages (average steps per hour, hours 0–23).
The function should be called **hourly_averages.** It takes `week_data` as an argument, and it returns a list of 7 lists, where each inner list contains 24 floats representing the average steps in each hour (0-23) for that day.

6. Data Output:

Create a function called **write_analysis_csv** that writes the analysis results to a CSV file. The function should take one argument, `analysis_dict,` which is the dictionary returned by the **smartwatch_analyzer** function, containing all the analysis results (`daily_analysis` and `peak_activity`).
The function must always write to the file named "analysis_report.csv"

The CSV file should have two sections:

Section 1: Daily Analysis

- Header row: `day, best_hour_start, best_hour_steps, morning_total, evening_total, winner, hour_0, hour_1, ..., hour_23`
- One row for each day (0-6) with the corresponding values
- The hourly averages should be the 24 values from the **hourly_averages** function

Section 2: Peak Activity

- Header row: `day, minute, steps, subseq_0, subseq_1, ..., subseq_14`

- One row with the peak day, peak minute, peak steps, and the 15-minute subsequence values

The function should:

- Include section headers ("daily analysis" and "peak activity")
- Add blank lines between sections for readability
- Save the file with the specified filename

7. Data filtering:

Create a function called **filter_high_activity_data** that processes step data while preserving the original. The function should:

- take two arguments: `week_data` and a `threshold` which is a minimum step count to keep (step count values below this become 0)
- Create a working copy for filtering operations
- Filter the data by setting the step count to 0 in minutes with steps below the threshold
- Return the filtered data
- This filtered data is not included in the CSV file above (but could be used in a more elaborate application), nor is it included in the Final Report dictionary.

Final Report

The main analysis function **smartwatch_analyzer** returns a final report in the form of a dict which contains:

- For each day: the day index (0–6) as 'day', the best hour start index and total, the morning/evening totals and winner, the 24 hourly averages.
- Peak activity includes the day index, the peak minute index, the step count at that minute, and the 15-minute subsequence.

The dict should be structured as in the following example, where the numerical values will depend on the input csv file but otherwise the data structure is given and fixed.

```
{ 'daily_analysis': [
        {
            'day': 0,
            'best_hour': (420, 1250),  # (start_minute, total_steps)
            'morning_evening': (850, 920, 'evening'),
                      # (morning_total, evening_total, winner)
            'hourly_avgs': [2.1, 1.8, 1.2, ..., 3.4]  # 24 values
```

```
        },
        # ... repeat for days 1-6
    ],
    'peak_activity': (0, 742, 38, [18, 16, 28, 27, 16, 19, 11, 38, 19,
27, 30, 24, 12, 16, 15])
    # (day, minute, steps, subsequence)
}
```

Note: please round all numeric results to one decimal place (e.g., 3.1).

Grading & Rubric

Test cases

The test cases total to 100 points. To get full marks, you must pass all the test cases. Some tests are given to you (public), and you will receive feedback on whether or not you passed them, but you won't know if you passed the private tests until we grade your assignment.

Style marks

You must satisfy various style conditions. Style points will be subtracted from your test case grade as follows:

- **-1: complicated and/or unreadable expressions**
- **-1: very long lines**

  One should not have to scroll horizontally to read the code. The commonly recommended maximum line length in Python is 79 characters, according to PEP 8, which is the Python Enhancement Proposal that provides the official style guide for writing Python code.

- **-1: too many or too few comments**

Place comments sparingly, e.g. when there is a complicated piece of logic. This will help you when you take a break from your code and return to it later. It will also help the T.A. if they need to understand your code.

Don't put comments for things that are obvious e.g. explain basic features of the Python language.

- **-1: unnecessary global variables**
- **-1: long or unnecessary (chain of) conditional statements.**
- **-1: comparing boolean expressions to True or False**
- **-1: redundant variables**

  Don't introduce variables that aren't needed, unless they represent a computed value that should be named so that the code is more clear.

- **-1: too much vertical spacing**

  We will only take the point off in extreme cases for this and the following one

- **-1: no vertical spacing where it would be helpful**

  We will also take points off for the following issues:

- **Up to -20: used material not seen in videos or in class activities unless explicitly authorized**

- **-15: submission had to be manually fixed and re-uploaded to correct a serious error**

  Think of this as an admin fee.

- -**5: otherwise not following assignment instructions**